

Mobile-Efficient Image Super-Resolution with Plain-Endpoint ECB Networks

Yuebo Hu

Abstract—Mobile single-image super-resolution (SR) must deliver both reconstruction quality and on-device latency. High-capacity models such as SwinIR (2022 SR SOTA) require computational power far beyond phone deployment. The balance is achieved by ECBSR-style networks, which train multi-branch Edge-oriented Convolution Blocks (ECBs) and fold them into a dense 3×3 convolution at inference. Our work explores an alternative Pareto choice to trade their structural robustness for further latency reduction. We retain the ECB computation budget and propose PECSR (Plain-Endpoint ECB Super-Resolution), which uses plain 3×3 endpoints and removes the global residual operation. Under a shared $4\times$ bicubic recipe and NCNN Vulkan export, PECSR is 14% faster in phone median latency at a 0.51% average-PSNR cost relative to standard ECBSR.

Index Terms—image super-resolution, mobile AI, efficient networks, knowledge distillation, on-device latency

I. INTRODUCTION

Single-image super-resolution (SR) recovers a high-resolution (HR) image from a low-resolution (LR) input. On a smartphone the same model must achieve satisfactory reconstruction within an interactive latency budget. Benchmark PSNR and FLOPs are standard metrics for quality and efficiency, but they do not necessarily predict whether a network deploys well on device.

The gap between FLOPs and measured speed is familiar in efficient vision [1]–[4]. Two graphs with similar FLOPs can still differ once head and tail operators, residual wiring, or export layout change. We therefore train $4\times$ bicubic SR models using MSE loss but compare them on actual phone latency besides desktop metrics (PSNR/SSIM/LPIPS).

In order to expand training capacity while keeping the inference structure simple, existing methods including RepVGG [2] and ECBSR [3] use multi-branch blocks inside the network and fold them into a dense 3×3 convolution at inference. ECBSR-M10C16 places Edge-oriented Convolution Blocks (ECBs), 10 in body, 1 in head, and 1 in tail. Each ECB sums parallel branches of plain 3×3 convolution, expand-squeeze convolution, and Sobel/Laplacian edge operators before folding them into one dense 3×3 . The network then adds a global LR residual before PixelShuffle [5], which we remove to reduce mobile latency. We also replace head and tail ECBs with plain 3×3 convolutions, and thus we name our model PECSR (Plain-Endpoint ECB Super-Resolution).

Fig. 1 compares PSNR and mobile latency measured on LR 180×180 images. Fig. 2 shows one qualitative comparison. Relative to FSRCNN (Fast Super-Resolution CNN) [6], a standard baseline for mobile SR, PECSR improves average PSNR at much lower FLOPs and lower phone latency. At

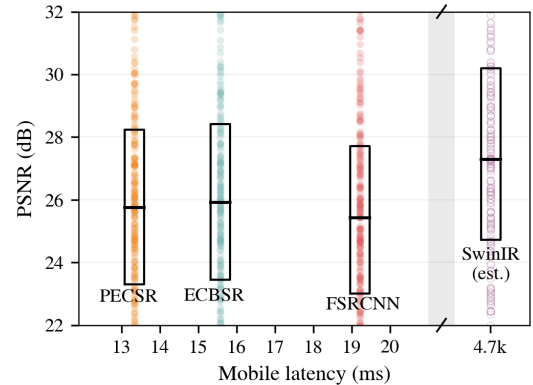


Fig. 1. Comparison between our PECSR and baseline models. Quartiles are labeled, but whiskers are out of range. Y axis is Per-image Y-PSNR on native Set5+Set14+BSD100+Urban100 LRs, and X axis is NCNN Vulkan FP16 median latency at fixed audit LR 180×180 . SwinIR phone cost is estimated, but its FLOPs are measured. Unweighted averages are in Tables IV and V.



Fig. 2. A qualitative comparison on BSD100 image 011, center crop. PECSR and ECBSR are close in quality, better than FSRCNN but not matching SwinIR. More comparisons are shown in Appendix B.

a shared ~ 0.98 G Conv-MAC budget, PECSR and ECBSR remain close in average PSNR, while PECSR records a lower phone median latency. An ablation under the PECSR shell shows that the multi-branch ECB trunk remains a major quality driver at fixed deploy cost (Sec. IV-C).

Overall, our PECSR improves over FSRCNN on quality at much lower FLOPs and lower phone latency, and trades a small average-PSNR gap to ECBSR for lower phone median latency. We explore the difference between FLOPs and mobile latency, and thus identify a new Pareto frontier of quality-latency trade-off that shows how a simplified network can be customized to pursue latency performance.

II. RELATED WORK

a) Transformer-based SR models: Transformer-based SR models have high quality but are far too costly for real-time mobile deployment. 2022 SOTA SwinIR [7] brought shifted-window Transformers into restoration and remains a widely used classical offline $\times 4$ SR model with public

checkpoints. We choose SwinIR as the offline quality reference and distillation teacher. In 2023, HAT [8] mixes window and channel attention to activate more pixels for reconstruction, SRFormer [9] lowers attention cost with permuted self-attention, and DAT [10] aggregates spatial and channel features for stronger detail recovery. In 2025, PFT [11] reports new benchmark peaks with progressive focused attention.

b) CNN-based SR models: CNN-based models focus more on efficiency, but most are still inapplicable for real-time mobile deployment. FSRCNN [6] maps in LR space with a transposed-convolution head. ESPCN [5] replaces explicit HR filtering with sub-pixel shuffle. CARN [12] cascades residual blocks for a lighter accuracy–cost trade-off. SESR [4] collapses over-parameterized linear blocks at inference and reports measured on-device cost beyond FLOPs alone.

c) Mobile operators and structures: On phones, operator choice and deploy-graph form matter besides FLOPs. Depthwise-separable convolution and ReLU6 [1] cut FLOPs and bound activations, yet measured latency still depends on layout, fusion, precision, and backend support. Structural re-parameterization, especially ECBSR [3], keeps a multi-branch training graph for capacity, then fold it into a plain inference graph for deployment, which originates from RepVGG [2], folding identity, 1×1 , and 3×3 branches into dense 3×3 stacks for classification. We take fused ECBSR as the matched-FLOPs dense baseline.

d) Distillation for compact SR: Distillation lets a large offline model supervise a compact deployable network without shipping the teacher. IMDN and RFDN [13], [14] show that feature distillation can improve the quality–cost curve of lightweight CNNs, but they embed distillation inside block design rather than transferring from an external teacher. We keep PECSR’s fused deploy graph fixed and instead borrow the loss view from perceptual SR of matching pixels or deep features, following perceptual loss [15] and VGG features [16]. We formulate those two objectives into PECSR in Sec. III-D and screen them at fixed fused deploy cost in Sec. IV-E.

III. METHOD

A. PECSR

As illustrated in Fig. 3, we modify the architecture of ECBSR (Edge-oriented Convolution Blocks for Super-Resolution) [3] to build PECSR (Plain-Endpoint ECB Super-Resolution). Relative to ECBSR, PECSR shares body width, depth, and fused Conv-MAC budget, omits the global LR residual, and replaces head/tail ECBs with plain 3×3 convolutions. Omitting the residual removes channel-repeat and pre-PixelShuffle add traffic that Conv-MAC accounting ignores, which helps on-device latency. Plain endpoints follow the common SR split of a shallow feature lift and a reconstruction projection [7], [17]: head maps $3 \rightarrow C$ and tail maps $C \rightarrow 3 \cdot 4^2$, where we replace ECBs with plain 3×3 convolutions. The body keeps $C \rightarrow C$ ECBs where identity shortcuts and full multi-branch re-parameterization apply. Let I_{LR} denote the low-resolution RGB input and $\hat{I}_{SR}$ the reconstructed high-resolution image. PECSR maps I_{LR} through a plain convo-

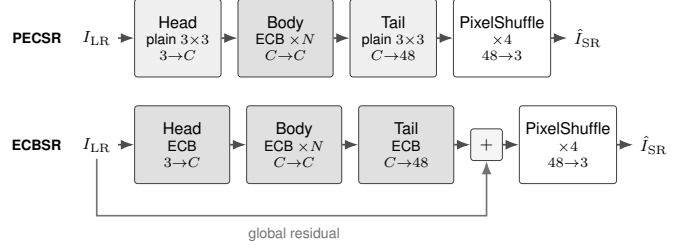


Fig. 3. PECSR vs. ECBSR network structures. PECSR uses plain 3×3 endpoints and omits the global LR residual; both fold each ECB to a dense 3×3 at inference.

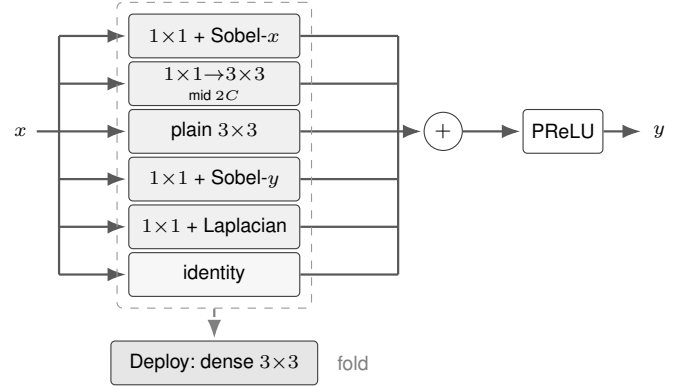


Fig. 4. ECB internal structure. Trained with six parallel branches, deployed with a folded dense 3×3 block that is numerically equal.

lution head, a stack of ECBs, and a plain convolution tail, then upsamples once with PixelShuffle:

$$\hat{I}_{SR} = \text{PixelShuffle}_{\times 4}(\text{Tail}(\text{ECBs}(\text{Head}(I_{LR}))))). \quad (1)$$

Here Head and Tail are plain 3×3 convolutions. The body is N equal-channel ECBs with PReLU, identity shortcuts, and expand–squeeze depth multiplier 2.0 (mid-channels $2C$ on the $1 \times 1 \rightarrow 3 \times 3$ branch). There is no global channel-repeat residual from I_{LR} into the pre-PixelShuffle tensor. Training uses the multi-branch ECB graph. Evaluation and export fold each ECB to a dense 3×3 . After folding, the fused and multi-branch graphs are numerically identical.

B. ECBSR baseline

We implement ECBSR following Zhang et al. [3] and the BasicSR reference. Fig. 4 shows the internal structure of one ECB block. For RGB, ECBSR adds a global residual into the pre-PixelShuffle tensor, so the three LR channels are repeated to $3 \cdot 4^2$ channels and added to the network prediction, which is algebraically equivalent to a nearest-neighbor upsample of I_{LR} after PixelShuffle. Train-time ECB is multi-branch ($\sim 149K$ parameters). Evaluation and export fold each block to a dense 3×3 ($\sim 31K$ parameters, 0.98 G FLOPs at LR 180×180). The fused graphs and multi-branch graphs are identical.

C. FLOPs

FLOPs denote multiply–accumulate counts (MAC) of convolutions at LR 180×180 . Let C be channel count and

N be body ECB count. For each dense 3×3 , FLOPs = $C_{\text{out}}C_{\text{in}}9HW$, summed over convolutions (PReLU, PixelShuffle, and residual adds are excluded). The fused PECSR stack has $N+2$ convolutions ($3 \rightarrow C$, N times $C \rightarrow C$, then $C \rightarrow 3 \cdot 4^2$), which expands to $\text{FLOPs}(C, N) = HW(27C + 9NC^2 + 432C)$ ($H=W=180$). ECBSR at $C=16$, $N=10$ sits near 0.98 G under this Conv-only accounting, so we adopt that pair as the shared fused budget ($\sim 31\text{K}$ parameters, twelve convolutions), and attribute later quality–latency differences mainly to the outer shell.

D. Distillation losses

We distill a frozen SwinIR classical $\times 4$ teacher into PECSR. Let I_{out} be the student output, I_{HR} the ground-truth HR image, and I_{teacher} the teacher output on the same LR input. Let L_{Charb} denote a Charbonnier (pseudo-Huber) loss [18] on RGB tensors, and let $\lambda \geq 0$ be the distillation weight. Following perceptual SR [15], [16], we consider pixel matching and VGG feature matching:

$$L = L_{\text{Charb}}(I_{\text{out}}, I_{\text{HR}}) + \lambda L_{\text{Charb}}(I_{\text{out}}, I_{\text{teacher}}) \quad (2)$$

$$L = L_{\text{Charb}}(I_{\text{out}}, I_{\text{HR}}) + \lambda \|\phi_3(I_{\text{out}}) - \phi_3(I_{\text{teacher}})\|_1 \quad (3)$$

where ϕ_3 denotes frozen VGG16 relu3 feature maps [16]. The two KD terms live on different scales, so λ is method-local rather than shared. Pixel Charbonnier acts on $[0, 1]$ RGB like the GT term, and we use $\lambda=0.2$ so distillation remains a moderate fraction of reconstruction. VGG relu3 L_1 acts on high-dimensional feature maps whose magnitude is much larger, so a smaller weight is required for the same balance [15], and we use $\lambda=0.01$. Each $\lambda>0$ setting is compared only to a matched $\lambda=0$ control under the same KD method.

IV. EXPERIMENTS

A. Setup

a) Datasets and metrics: We study classical $4 \times$ bicubic SR. Training uses DIV2K [19]: 256×256 HR crops from the 800-image train split, downsampled on-the-fly with PIL bicubic to 64×64 LR patches. Augmentation uses random flips and 90° rotations, with pixels in $[0, 1]$ RGB. Checkpoints are selected by RGB tensor PSNR on DIV2K validation. Final tables report Y-channel PSNR/SSIM with a 4-pixel border shave on Set5, Set14, BSD100, and Urban100, plus RGB LPIPS (AlexNet) [20], [21].

b) Training recipe: Architecture comparisons use MSE with Adam ($\text{lr}=10^{-3}$), batch size 24, MultiStepLR, seed 42, AMP, and $\sim 20,000$ optimizer steps; FSRCNN, ECBSR, and PECSR share the same update count. With 800 train images and the last incomplete batch dropped, batch size 24 yields $\text{steps/epoch} = \lfloor 800/24 \rfloor$ and $\text{epochs} = \lceil 20000/\text{steps/epoch} \rceil$, i.e. 607 epochs. The learning rate decays by $\gamma=0.5$ at $\sim 60\%$ / 85% of that schedule (epochs 364 and 516). Unless noted, quality and phone entries use the 20k-step checkpoint with best RGB PSNR on the first 100 DIV2K validation images.

TABLE I
BENCHMARK QUALITY UNDER FAIR BUDGET. AVERAGE IS AN UNWEIGHTED MEAN OVER FOUR DATASETS. [†]PUBLIC PRETRAINED.

Model	PSNR	SSIM	LPIPS
FSRCNN	26.81	0.744	0.176
ECBSR	27.51	0.768	0.144
PECSR	27.37	0.764	0.156
SwinIR [†]	29.13	0.814	0.108

TABLE II
PARAMS / FLOPs (LR 180×180) AND DESKTOP MEDIAN LATENCY (RTX 4060, CUDA EVENTS).

Model	Params	FLOPs (G)	FP32 (ms)	FP16 (ms)
FSRCNN	25K	7.41	1.24	1.70
ECBSR	31K	0.98	1.59	1.61
PECSR	31K	0.98	1.40	1.46
SwinIR [†]	11.9M	417.8	600.6	458.7

c) Baselines: FSRCNN [6], ECBSR, and PECSR are trained under the shared recipe above. FSRCNN is the classical compact baseline ($\sim 25\text{K}$ parameters, 7.41 G FLOPs at LR 180×180) with a 9×9 transposed-convolution upsampler. ECBSR is the standard ECB structure baseline. SwinIR [7] classical $\times 4$ ($\sim 11.9\text{M}$ parameters, 417.8 G FLOPs at LR 180×180) is an offline quality reference and the distillation teacher. We use the authors’ public pretrained classical $\times 4$ checkpoint and do not retrain SwinIR. A rough phone estimate of SwinIR’s phone latency $\sim 4.7\text{k}$ ms scales desktop SwinIR FP16 timing by the measured FSRCNN phone-to-desktop FP16 ratio.

d) Latency and deployment: FLOPs, desktop CUDA latency, and phone latency all use LR 180×180 (HR 720×720 after $\times 4$). Desktop latency uses PyTorch CUDA events on an RTX 4060 Laptop GPU, batch 1, 100 warmup / 500 timed iterations. Phone measurement uses NCNN Vulkan FP16 on a Vivo phone (Dimensity 9400 SoC), 50 warmup / 300 timed iterations. Export follows PyTorch to PNNX to NCNN.

B. Quality and efficiency comparisons

Tables I–II summarize the main comparison. PECSR reaches 27.37 dB average PSNR, improving over FSRCNN by $+0.56$ dB average PSNR at $7.6 \times$ lower FLOPs, with higher average SSIM and lower average LPIPS (Table I). PECSR is 0.14 dB (0.51%) below ECBSR at a shared ~ 0.98 G Conv-MAC budget, but recording 2.24 ms (14%) lower phone median latency (Table IV). Per-dataset PSNR and three-seed confirmation are in Appendix A.

C. Ablations

PECSR and ECBSR share an ECB body and differ in the outer shell. We study the body’s contribution to quality under a fixed PECSR shell by replacing the body with plain 3×3 convolutions, and keep same $C=16$, $N=10$.

TABLE III
BODY ABLATION UNDER THE PECSR SHELL. PLAIN-BODY KEEPS PLAIN
ENDPOINTS AND NO GLOBAL RESIDUAL.

Model	Train body	PSNR	SSIM	LPIPS
Plain-body	3×3	27.02	0.752	0.174
PECSR	ECB	27.37	0.764	0.156
ECBSR	ECB	27.51	0.768	0.144

TABLE IV
QUALITY-COST SUMMARY. PSNR: UNWEIGHTED FOUR-DATASET
AVERAGE (NATIVE TEST LRS). FLOPS AND PHONE LATENCY: AUDIT LR
180×180. [†]PUBLIC PRETRAINED. [‡]ESTIMATED PHONE LATENCY.

Model	PSNR	FLOPs	Phone Latency
FSRCNN	26.81	7.41	19.22
ECBSR	27.51	0.98	15.57
PECSR	27.37	0.98	13.33
SwinIR [†]	29.13	417.8	~4.7k [‡]

As is shown in Table III, the plain-body variant loses 0.35 dB average PSNR and worsens average LPIPS relative to PECSR. The gap exceeds the late-training validation fluctuation of the plain run ($2 \times \text{std}$ of the last 20 epochs ≈ 0.11 dB). Therefore, the ablation confirms that the multi-branch ECB training body is indispensable for quality.

D. FLOPs, backend, and phone latency

Table IV summarizes average PSNR with the same 180×180 phone medians used on the latency axis of Fig. 1.

a) Desktop: On RTX 4060, PECSR and ECBSR share a ~ 0.98 G Conv-MAC budget while PECSR records lower median latency in both FP32 and FP16. FSRCNN has much higher FLOPs; its desktop FP32 median is slightly lower than PECSR while FP16 is higher, so FLOPs remain an incomplete indicator of deployable speed. SwinIR’s desktop medians (600.6 ms FP32 / 458.7 ms FP16) already exceed a full frame budget on a discrete GPU, so they are used for estimation rather than compared with real measurements.

b) Phone: Table V reports NCNN Vulkan FP16 latency on Dimensity 9400 at LR 180×180. FSRCNN uses a single measurement session; PECSR and ECBSR report means over three sessions. FSRCNN (19.22 ms median) remains slower despite its different operator stack.

E. Distillation

Using the losses in Sec. III-D, we screen pixel and VGG relu3 distillation into PECSR with short $\sim 2k$ -update Charbonnier runs at fixed fused deploy cost (Table VI). Each $\lambda > 0$ setting is compared only to its matched $\lambda = 0$ control; scores are not comparable to the 20k-step MSE table (Table I).

Pixel matching at $\lambda = 0.2$ raises average PSNR by +0.51 dB over its control, with a small LPIPS gain and a small SSIM drop. VGG relu3 at $\lambda = 0.01$ does not improve average PSNR (−0.13 dB). Under this short protocol, pixel matching helps while the tested feature-KD setting does not.

TABLE V
ON-DEVICE PERFORMANCE AT LR 180×180. MED AND P90 ARE MEDIAN
AND 90TH PERCENTILE LATENCY. NCNN IN BINARY SIZE (MB).

Model	FLOPs	Med	p90	NCNN
FSRCNN	7.41	19.22	21.71	0.10
ECBSR	0.98	15.57	17.31	0.12
PECSR	0.98	13.33	15.93	0.12

TABLE VI
DISTILLATION INTO PECSR UNDER A $\sim 2k$ -UPDATE BUDGET. EACH $\lambda > 0$
RUN IS COMPARED ONLY TO ITS MATCHED $\lambda = 0$ CONTROL. AVERAGES
ARE UNWEIGHTED OVER FOUR DATASETS.

KD	λ	PSNR	SSIM	LPIPS	Δ PSNR
Pixel	0	24.46	0.723	0.224	—
Pixel	0.2	24.97	0.711	0.216	+0.51
VGG relu3	0	24.47	0.721	0.224	—
VGG relu3	0.01	24.34	0.724	0.225	−0.13

V. CONCLUSION AND FUTURE WORK

Mobile SR is decided jointly by reconstruction quality and on-device latency. Our results show that the outer shell around a ECB trunk is a deploy-relevant design axis. PECSR keeps a ~ 0.98 G Conv-MAC budget with a slightly lighter residual-free shell, records 14% lower phone latency at the cost of a small 0.51% PSNR gap, and improves over FSRCNN on both quality and phone speed. A body ablation proves multi-branch ECB trunk contributes to quality. Distillation does not improve quality under the tested VGG setting.

These conclusions are scoped to classical $4 \times$ bicubic SR, one NCNN Vulkan handset, and Conv-FLOPs accounting that ignores memory traffic. Future work can apply the Pareto front to other quantization methods and SoC devices. A CPU-specific study on each layer of the network can further push the crossed frontiers of SR architecture and mobile deep learning, of computer vision and edge computing.

REFERENCES

- [1] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [2] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets great again," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 13 733–13 742, verified 2026-07-10 against CVF Open Access. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2021/html/Ding_RepVGG_Making_VGG-Style_ConvNets_Great_Again_CVPR_2021_paper.html
- [3] X. Zhang, H. Zeng, and L. Zhang, "Edge-oriented convolution block for real-time super resolution on mobile devices," in *Proceedings of the 29th ACM International Conference on Multimedia (ACM MM)*, October 2021, pp. 4034–4043, verified 2026-07-10 against ACM DL + author PDF/GitHub bibtex. [Online]. Available: <https://dl.acm.org/doi/10.1145/3474085.3475291>
- [4] K. Bhardwaj, M. Milosavljevic, L. O'Neil, D. Gope, R. Matas, A. Chalfin, N. Suda, L. Meng, and D. Loh, "Collapsible linear blocks for super-efficient super resolution," in *Proceedings of Machine Learning and Systems (MLSys)*, 2022, sESR; verified against MLSys 2022 proceedings. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2022/hash/3134f61af2136e249b0d8f190cbdc508-Abstract.html
- [5] W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016, pp. 1874–1883. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/Shi_Real-Time_Single_Image_CVPR_2016_paper.html
- [6] C. Dong, C. C. Loy, and X. Tang, "Accelerating the super-resolution convolutional neural network," in *Proceedings of the European Conference on Computer Vision (ECCV)*, October 2016, pp. 391–407. [Online]. Available: <https://mlanthology.org/eccv/2016/dong2016eccv-a ccelerating/>
- [7] J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, and R. Timofte, "SwinIR: Image restoration using Swin Transformer," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, October 2021, pp. 1833–1844, advances in Image Manipulation (AIM) Workshop. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2021W/AIM/html/Liang_SwinIR_Image_Restoration_Using_Swin_Transformer_ICCVW_2021_paper.html
- [8] X. Chen, X. Wang, J. Zhou, Y. Qiao, and C. Dong, "Activating more pixels in image super-resolution transformer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 22 367–22 377, hAT; verified 2026-07-12 against CVF Open Access. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2023/html/Chen_Activating_More_Pixels_in_Image_Super-Resolution_Transformer_CVPR_2023_paper.html
- [9] Y. Zhou, Z. Li, C.-L. Guo, S. Bai, M.-M. Cheng, and Q. Hou, "SRFormer: Permuted self-attention for single image super-resolution," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 12 780–12 791, verified 2026-07-12 against CVF Open Access. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2023/html/Zhou_SRFormer_Permuted_Self-Attention_for_Single_Image_Super-Resolution_ICCV_2023_paper.html
- [10] Z. Chen, Y. Zhang, J. Gu, L. Kong, X. Yang, and F. Yu, "Dual aggregation transformer for image super-resolution," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 12 278–12 287, dAT; verified 2026-07-12 against CVF Open Access. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2023/html/Chen_Dual_Aggregation_Transformer_for_Image_Super-Resolution_ICCV_2023_paper.html
- [11] W. Long, X. Zhou, L. Zhang, and S. Gu, "Progressive focused transformer for single image super-resolution," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 2279–2288, pFT; verified 2026-07-12 against CVF Open Access. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2025/html/Long_Progressive_Focused_Transformer_for_Single_Image_Super-Resolution_CVPR_2025_paper.html
- [12] N. Ahn, B. Kang, and K.-A. Sohn, "Fast, accurate, and lightweight super-resolution with cascading residual network," in *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018, pp. 252–268. [Online]. Available: https://openaccess.thecvf.com/content_ECCV_2018/html/Namhyuk_Ahn_Fast_Accurate_and_ECCV_2018_paper.html
- [13] Z. Hui, X. Gao, Y. Yang, and X. Wang, "Lightweight image super-resolution with information multi-distillation network," in *Proceedings of the 27th ACM International Conference on Multimedia (ACM MM)*, October 2019, pp. 2024–2032. [Online]. Available: <https://dl.acm.org/doi/10.1145/3343031.3351084>
- [14] J. Liu, J. Tang, and G. Wu, "Residual feature distillation network for lightweight image super-resolution," in *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*, August 2020, pp. 41–55, aIM 2020 Efficient Super-Resolution Workshop. [Online]. Available: <https://mlanthology.org/eccvw/2020/liu2020eccvw-residual/>
- [15] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual losses for real-time style transfer and super-resolution," in *Proceedings of the European Conference on Computer Vision (ECCV)*, October 2016, pp. 694–711. [Online]. Available: <https://mlanthology.org/eccv/2016/johnson2016eccv-perceptual/>
- [16] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [17] B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, "Enhanced deep residual networks for single image super-resolution," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, July 2017, pp. 1132–1140. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2017_workshops/w12/papers/Lim_Enhanced_Deep_Residual_CVPR_2017_paper.html
- [18] J. T. Barron, "A general and adaptive robust loss function," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019, pp. 4331–4339, charbonnier / pseudo-Huber is a special case; used here for the distillation RGB term. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/html/Barron_A_General_and_Adaptive_Robust_Loss_Function_CVPR_2019_paper.html
- [19] E. Agustsson and R. Timofte, "NTIRE 2017 challenge on single image super-resolution: Dataset and study," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, July 2017, pp. 126–135. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2017_workshops/w12/html/Agustsson_NTIRE_2017_Challenge_CVPR_2017_paper.html
- [20] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018, pp. 586–595. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Zhang_The_Unreasonable_Effectiveness_CVPR_2018_paper.html
- [21] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, April 2004.

APPENDIX A

ADDITIONAL QUANTITATIVE RESULTS

Table VII breaks down the seed-42 Y-channel PSNR behind Table I. Table VIII retrain PECSR and ECBSR at seeds 123 and 2026 under the same recipe (mean $\pm$ sample std over three seeds). The mean PECSR–ECBSR gap matches the seed-42 result, and the seed standard deviation is smaller than that gap, so the quality ordering is stable. Phone latency in Sec. IV-D uses the seed-42 checkpoints.

TABLE VII
PER-DATASET Y-CHANNEL PSNR (dB).

Model	Set5	Set14	BSD100	Urban100
FSRCNN	29.76	27.00	26.59	23.90
ECBSR	30.70	27.68	26.99	24.66
PECSR	30.50	27.58	26.92	24.49

TABLE VIII
THREE-SEED CONFIRMATION (SEEDS 42, 123, 2026). ENTRIES ARE MEAN $\pm$ SAMPLE STD OF THE UNWEIGHTED FOUR-DATASET AVERAGE.

Model	PSNR	SSIM	LPIPS
ECBSR	27.57 $\pm$ 0.06	0.770 $\pm$ 0.002	0.142 $\pm$ 0.003
PECSR	27.42 $\pm$ 0.05	0.765 $\pm$ 0.001	0.154 $\pm$ 0.002

APPENDIX B

QUALITATIVE COMPARISONS

Additional crops with the same column layout as Fig. 2.



Fig. 5. Urban100 image 001, bottom-left crop.

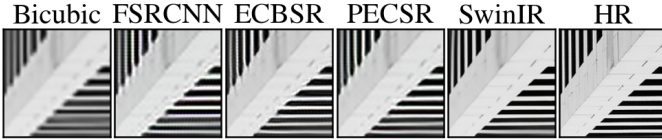


Fig. 6. Urban100 image 011, bottom-left crop.

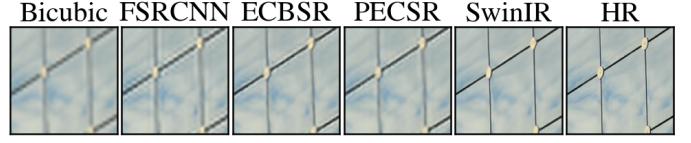


Fig. 7. Urban100 image 029, top-right crop.



Fig. 8. BSD100 image 036, center crop.



Fig. 9. BSD100 image 046, top-left crop.



Fig. 10. BSD100 image 051, top-left crop.